

EXPERIMENT 1

Objective: To implement stack operations with overflow and underflow handling, convert an infix expression to postfix using operator precedence, and evaluate the postfix expression using a stack.

Stack Implementation

```
#include <bits/stdc++.h>
using namespace std;

int s[100];
int n = 100;
int i = 0;

void push() {
    if (i == n) {
        cout << "Stack overflow" << endl;
        return;
    }
    int input;
    cin >> input;
    s[i] = input;
    i++;
}

void pop() {
    if (i == 0) {
        cout << "Stack underflow" << endl;
        return;
    }
    i--;
    s[i] = -1;
}

void peek() {
    if (i == 0) {
        cout << "Stack is empty" << endl;
        return;
    }
}
```

```
    i--;  
    cout << s[i] << " ";  
    i++;  
}
```

```
void display () {  
    for (int j=n-1; j>=0; j--) {  
        if (s[j] != -1) {  
            cout << s[j] << " ";  
        }  
    }  
    cout << endl;  
}
```

```
int main () {  
    for (int j=0; j<n; j++) {  
        s[j] = -1;  
    }
```

```
    int choice;  
    cout << "Enter your choice (1: push, 2: pop, 3: peek, 4: display, 0: exit): ";  
    cin >> choice;
```

```
    while (choice != 0) {  
        switch (choice) {  
            case 1:  
                push();  
                break;  
            case 2:  
                pop();  
                break;  
            case 3:  
                peek();  
                break;  
            case 4:  
                display();  
                break;  
            default:  
                cout << "Invalid choice" << endl;  
        }
```

```
    cout << "Enter your choice (1: push, 2: pop, 3: peek, 4: display, 0: exit): ";
```

```

        cin >> choice;
    }
    return 0;
}

```

Output:

```

PS C:\Second Year\3rd Semester\Advanced DSA using C++\File Experiments> cd "c:\Second Year\3rd Semester\Advanced DSA using C++\" ; if ($?) { g++ Stack.cpp -o Stack } ; if ($?) { .\Stack }
Enter your choice (1: push, 2: pop, 3: peek, 4: display, 0: exit): 1
45
Enter your choice (1: push, 2: pop, 3: peek, 4: display, 0: exit): 1
45
Enter your choice (1: push, 2: pop, 3: peek, 4: display, 0: exit): 4
45 45
Enter your choice (1: push, 2: pop, 3: peek, 4: display, 0: exit): 2
Enter your choice (1: push, 2: pop, 3: peek, 4: display, 0: exit): 4
45
Enter your choice (1: push, 2: pop, 3: peek, 4: display, 0: exit):

```

Infix to Postfix

```

#include <bits/stdc++.h>
using namespace std;

```

```

char st[100];
int m = 100;
int i = 0;

```

```

void push(char x) {
    st[i] = x;
    i++;
}

```

```

void pop() {
    i--;
    st[i] = -1;
}

```

```

void top() {
    if (i > 0) {
        i--;
        cout << st[i] << " ";
    }
}

```

```
}
```

```
void display () {
```

```
    for (int j=m-1; j>=0; j--) {
```

```
        if (st[j] != -1) {
```

```
            cout << st[j] << " ";
```

```
        }
```

```
    }
```

```
}
```

```
int main () {
```

```
    string infix;
```

```
    cout << "Enter infix expression (don't use spaces): ";
```

```
    cin >> infix;
```

```
    int n = infix.length();
```

```
    string postfix = "";
```

```
    for (int j=0; j<n; j++) {
```

```
        if (isalpha(infix[j])){
```

```
            postfix += infix[j];
```

```
        }
```

```
        else if (infix[j] == ')') {
```

```
            while (i > 0 && st[i-1] != '(') {
```

```
                postfix += st[i-1];
```

```
                i--;
```

```
            }
```

```
            if (i > 0) {
```

```
                i--;
```

```
            }
```

```
        }
```

```
        else if (infix[j] == '+' || infix[j] == '-' || infix[j] == '*' || infix[j] == '/' || infix[j] == '^' || infix[j] == '%') {
```

```
            push(infix[j]);
```

```
        }
```

```
    }
```

```
    while( i > 0) {
```

```
        postfix += st[i-1];
```

```
        i--;
```

```
    }
```

```
    cout << "Postfix Conversion: " << postfix << endl;
```

```
}
```

Output:

```
● PS C:\Second Year\3rd Semester\Advanced DSA using C++> cd "c:\Second Y
  ○ Infix } ; if ($?) { .\Infix }
    Enter infix expression (don't use spaces): (A+B)/C
    Postfix Conversion: AB+C/
  ○ PS C:\Second Year\3rd Semester\Advanced DSA using C++> █
```

Satyansh Yadav (2503201001031)

EXPERIMENT 2

Objective: To implement linear and circular queue operations using FIFO, handle overflow and underflow conditions, and display the final queue state.

Linear queue

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
int q[100];
```

```
int i = 0;
```

```
int j = 0;
```

```
int n = 100;
```

```
void pop() {
```

```
    if (i == j) {
```

```
        cout << "Queue underflow" << endl;
```

```
        return;
```

```
    }
```

```
    i++;
```

```
}
```

```
void push(int x) {
```

```
    if (j == n) {
```

```
        cout << "Queue overflow" << endl;
```

```
        return;
```

```
    }
```

```
    q[j] = x;
```

```
    j++;
```

```
}
```

```
void peek() {
```

```
    if (i == j) {
```

```
        cout << "Queue is empty" << endl;
```

```
        return;
```

```
    }
```

```
    cout << q[i] << endl;
```

```
}
```

```
void display() {  
    if (i == j) {  
        cout << "Queue is empty" << endl;  
        return;  
    }  
    for (int k = i; k < j; k++) {  
        cout << q[k] << " ";  
    }  
    cout << endl;  
}
```

```
int main() {  
    for (int k = 0; k < 100; k++) {  
        q[k] = -1;  
    }
```

```
    int choice;  
    cout << "Enter your choice (1: enqueue, 2: dequeue, 3: peek, 4: display, 0: exit): ";  
    cin >> choice;
```

```
    while (choice != 0) {  
        switch (choice) {  
            case 1: {  
                int x;  
                cout << "Enter element: ";  
                cin >> x;  
                push(x);  
                break;  
            }
```

```
            case 2:  
                pop();  
                break;
```

```
            case 3:  
                cout << "Front element: ";  
                peek();  
                break;
```

```
            case 4:  
                cout << "Queue: ";
```

```

        display();
        break;

    default:
        cout << "Invalid choice" << endl;
    }
    cout << "Enter your choice (1: enqueue, 2: dequeue, 3: peek, 4: display, 0: exit): ";
    cin >> choice;
}
return 0;
}

```

Output:

```

PS C:\Second Year\3rd Semester\Advanced DSA using C++\File Experiments> cd "c:\Second
ents\" ; if ($?) { g++ 1.cpp -o 1 } ; if ($?) { .\1 }
Enter your choice (1: enqueue, 2: dequeue, 3: peek, 4: display, 0: exit): 1
Enter element: 1
Enter your choice (1: enqueue, 2: dequeue, 3: peek, 4: display, 0: exit): 1
Enter element: 2
Enter your choice (1: enqueue, 2: dequeue, 3: peek, 4: display, 0: exit): 1
Enter element: 3
Enter your choice (1: enqueue, 2: dequeue, 3: peek, 4: display, 0: exit): 1
Enter element: 4
Enter your choice (1: enqueue, 2: dequeue, 3: peek, 4: display, 0: exit): 4
Queue: 1 2 3 4
Enter your choice (1: enqueue, 2: dequeue, 3: peek, 4: display, 0: exit): 3
Front element: 1
Enter your choice (1: enqueue, 2: dequeue, 3: peek, 4: display, 0: exit): 2
Enter your choice (1: enqueue, 2: dequeue, 3: peek, 4: display, 0: exit): 4
Queue: 2 3 4
Enter your choice (1: enqueue, 2: dequeue, 3: peek, 4: display, 0: exit): 

```

Circular queue

```

#include <bits/stdc++.h>
using namespace std;

```

```

int q[100];
int i = 0;
int j = 0;
int n = 100;
int cnt = 0;

```

```

void push(int x) {

```

```

    if (cnt == n) {
        cout << "Queue overflow" << endl;
        return;
    }
    q[j] = x;
    j = (j + 1) % n;
    cnt++;
}

void pop() {
    if (cnt == 0) {
        cout << "Queue underflow" << endl;
        return;
    }
    q[i] = -1;
    i = (i + 1) % n;
    cnt--;
}

void peek() {
    if (cnt == 0) {
        cout << "Queue is empty" << endl;
        return;
    }
    cout << q[i] << endl;
}

void display() {
    if (cnt == 0) {
        cout << "Queue is empty" << endl;
        return;
    }
    int k = i;
    for (int x = 0; x < cnt; x++) {
        cout << q[k] << " ";
        k = (k + 1) % n;
    }
    cout << endl;
}

```

```

int main() {
    for (int k = 0; k < 100; k++) {
        q[k] = -1;
    }

    int choice;
    cout << "Enter your choice (1: enqueue, 2: dequeue, 3: peek, 4: display, 0: exit): ";
    cin >> choice;

    while (choice != 0) {
        switch (choice) {
            case 1: {
                int x;
                cout << "Enter element: ";
                cin >> x;
                push(x);
                break;
            }

            case 2:
                pop();
                break;

            case 3:
                cout << "Front element: ";
                peek();
                break;

            case 4:
                cout << "Queue: ";
                display();
                break;

            default:
                cout << "Invalid choice" << endl;
        }
        cout << "Enter your choice (1: enqueue, 2: dequeue, 3: peek, 4: display, 0: exit): ";
        cin >> choice;
    }
    return 0;
}

```

Output:

```
PS C:\Second Year\3rd Semester\Advanced DSA using C++\File Experiments> cd "c:\S
ents\" ; if ($?) { g++ 1.cpp -o 1 } ; if ($?) { .\1 }
Enter your choice (1: enqueue, 2: dequeue, 3: peek, 4: display, 0: exit): 1
Enter element: 34
Enter your choice (1: enqueue, 2: dequeue, 3: peek, 4: display, 0: exit): 1
Enter element: 23
Enter your choice (1: enqueue, 2: dequeue, 3: peek, 4: display, 0: exit): 1
Enter element: 45
Enter your choice (1: enqueue, 2: dequeue, 3: peek, 4: display, 0: exit): 4
Queue: 34 23 45
Enter your choice (1: enqueue, 2: dequeue, 3: peek, 4: display, 0: exit): 2
Enter your choice (1: enqueue, 2: dequeue, 3: peek, 4: display, 0: exit): 3
Front element: 23
Enter your choice (1: enqueue, 2: dequeue, 3: peek, 4: display, 0: exit): 4
Queue: 23 45
Enter your choice (1: enqueue, 2: dequeue, 3: peek, 4: display, 0: exit):
```

Satyansh Yadav (25032)

EXPERIMENT 3

Objective: To implement recursive functions to calculate the factorial and Nth Fibonacci number without using iterative constructs.

Factorial

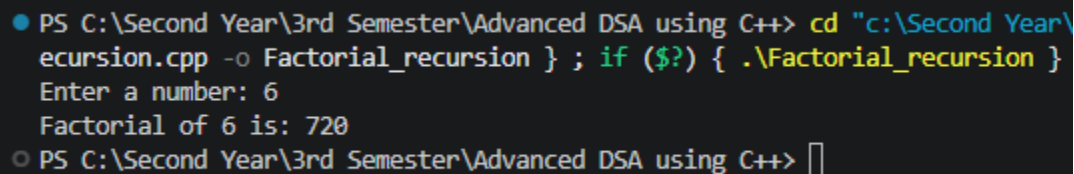
```
#include <bits/stdc++.h>
using namespace std;
```

```
int factorial (int n) {
    if (n == 0) {
        return 1;
    }
    else {
        return n*factorial(n-1);
    }
}

int main() {
    int n;
    cout << "Enter a number: ";
    cin >> n;

    int a = factorial(n);
    cout << "Factorial of " << n << " is: " << a << endl;
    return 0;
}
```

Output:



```
● PS C:\Second Year\3rd Semester\Advanced DSA using C++> cd "c:\Second Year\
ecursion.cpp -o Factorial_recursion } ; if ($?) { .\Factorial_recursion }
Enter a number: 6
Factorial of 6 is: 720
○ PS C:\Second Year\3rd Semester\Advanced DSA using C++> █
```

Fibonacci

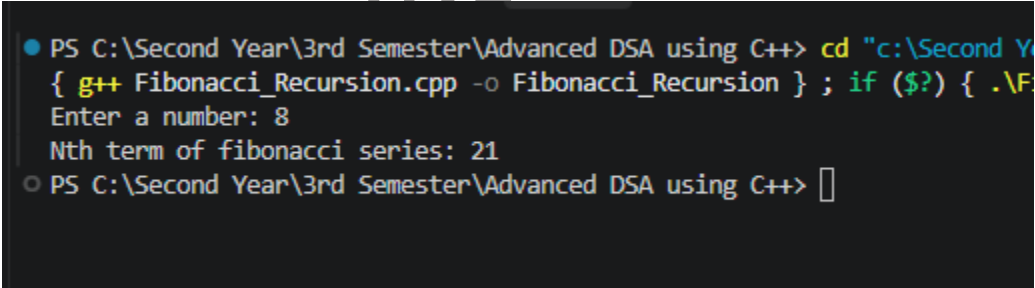
```
#include <iostream>
```

```
using namespace std;
```

```
int fibonacci(int n, int a = 0, int b = 1) {  
    if (n == 0) {  
        return a;  
    }  
    if (n == 1) {  
        return b;  
    }  
    return fibonacci(n - 1, b, a + b);  
}
```

```
int main() {  
    int n;  
    cout << "Enter a number: ";  
    cin >> n;  
  
    cout << "Nth term of fibonacci series: " << fibonacci(n) << endl;  
  
    return 0;  
}
```

Output:



```
PS C:\Second Year\3rd Semester\Advanced DSA using C++> cd "c:\Second Year\3rd Semester\Advanced DSA using C++>  
{ g++ Fibonacci_Recursion.cpp -o Fibonacci_Recursion } ; if ($?) { .\Fibonacci_Recursion.exe 8 }  
Enter a number: 8  
Nth term of fibonacci series: 21  
PS C:\Second Year\3rd Semester\Advanced DSA using C++> 
```

EXPERIMENT 4

Objective: To sort an array of N integers using a Divide and Conquer sorting technique such as Merge Sort or Quick Sort, and display the elements of the array in ascending order.

Merge Sort

```
#include <bits/stdc++.h>
using namespace std;
```

```
void merge(int arr1[], int arr2[], int size1, int size2, int arr[]) {
    int i = 0, j = 0, k = 0;
```

```
    while (i < size1 && j < size2) {
        if (arr1[i] < arr2[j]) {
            arr[k] = arr1[i];
            i++;
        }
        else {
            arr[k] = arr2[j];
            j++;
        }
        k++;
    }
```

```
    while (i < size1) {
        arr[k] = arr1[i];
        i++;
        k++;
    }
```

```
    while (j < size2) {
        arr[k] = arr2[j];
        j++;
        k++;
    }
}
```

```
void mergeSort(int arr[], int n) {
    if (n <= 1)
```

```

        return;

    int size1 = n / 2;
    int size2 = n - size1;

    int arr1[size1];
    int arr2[size2];

    for (int i = 0; i < size1; i++)
        arr1[i] = arr[i];

    for (int i = 0; i < size2; i++)
        arr2[i] = arr[size1 + i];

    mergeSort(arr1, size1);
    mergeSort(arr2, size2);

    merge(arr1, arr2, size1, size2, arr);
}

int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;

    int arr[n];
    cout << "Enter the elements: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    mergeSort(arr, n);
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    return 0;
}

```

Output:

```
PS C:\Second Year\3rd Semester\Advanced DSA using C++> cd '
cpp -o merge_sort } ; if ($?) { .\merge_sort }
Enter the number of elements: 7
Enter the elements: 755 435 45 522 585 22 5
Sorted array: 5 22 45 435 522 585 755
PS C:\Second Year\3rd Semester\Advanced DSA using C++> █
```

Quick Sort

```
#include <iostream>
```

```
using namespace std;
```

```
int partition(int arr[], int l, int h) {
```

```
    int pivot = arr[h];
```

```
    int i = l - 1;
```

```
    for (int j = l; j < h; j++) {
```

```
        if (arr[j] < pivot) {
```

```
            i++;
```

```
            int temp = arr[i];
```

```
            arr[i] = arr[j];
```

```
            arr[j] = temp;
```

```
        }
```

```
    }
```

```
    int temp = arr[i + 1];
```

```
    arr[i + 1] = arr[h];
```

```
    arr[h] = temp;
```

```
    return i + 1;
```

```
}
```

```
void quickSort(int arr[], int l, int h) {
```

```
    if (l < h) {
```

```
        int p = partition(arr, l, h);
```

```
        quickSort(arr, l, p - 1);
```

```

        quickSort(arr, p + 1, h);
    }
}

int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;

    int arr[n];
    cout << "Enter the elements: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    quickSort(arr, 0, n - 1);
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    return 0;
}

```

Output:

```

● PS C:\Second Year\3rd Semester\Advanced DSA using C++> cd "c:\
  cpp -o Quick_sort } ; if ($?) { .\Quick_sort }
  Enter the number of elements: 5
  Enter the elements: 34 22 -2 455 3
  Sorted array: -2 3 22 34 455
○ PS C:\Second Year\3rd Semester\Advanced DSA using C++> 

```